OpenPrism Network: A UMA-First Distributed Inference Architecture with Two Deployment Models

Abdalla Doleh* Ratna Babu Chinnam†
Industrial Engineering, Wayne State University, Detroit, MI, USA

May 30, 2026

Zenodo DOI: 10.5281/zenodo.20465112

Abstract

The energy and capital demand of large-language-model (LLM) inference is on a trajectory that dedicated GPU data centres cannot satisfy sustainably. We observe that Unified Memory Architecture (UMA) hardware—exemplified by Apple Silicon and Qualcomm Snapdragon X—delivers competitive inference performance per watt outside the very high batch regime, while existing distributed-compute networks (Gensyn, Bittensor, io.net, Akash) treat heterogeneous GPUs as the unit of work and regard neither UMA hardware nor energy efficiency as a first-class design constraint. We propose OpenPrism Network, a UMA-first distributed inference architecture in which transformer layers are *statically owned* by nodes so that weights remain resident and only activations transit the network; a blockchain layer is restricted to settlement, reputation, and payment and never to compute; output integrity is established by multi-node redundancy with tolerance-banded fingerprinting rather than zero-knowledge proofs; and routing is locality-aware, keeping inference within metro-area clusters. The network is explicitly scoped to batch- and throughput-oriented, latency-tolerant workloads. We describe two deployment models: a distributed mesh harvesting idle institutional capacity, and a purpose-built UMA micro data center deployable by resource-constrained organizations as a sovereign inference facility. This is a *position and architecture* paper: we claim no original experimental results, and all quantitative figures are drawn from publicly available benchmarks and published specifications, cited explicitly. We report performance per watt honestly—including the threefold cost of consensus—and find that UMA nodes *lose* on operational efficiency against batched data-centre GPUs in the scoped regime; the architecture's advantage is therefore established on capital, embodied energy, total cost of ownership, and data sovereignty, not on operational joules per token. We frame two problems as genuinely unsolved: a consensus protocol for ML output verification under floating-point non-determinism, and a dynamic layer-assignment protocol that rebalances ownership as nodes join and leave without full weight redistribution.

Contents

*Corresponding author. ORCID: 0009-0008-5192-2167. Email: ai5145@wayne.edu.
†Email: ai2396@wayne.edu.

1 Introduction

The compute and energy footprint of LLM inference has grown faster than the efficiency of the dedicated accelerators serving it. Inference, not training, now dominates the lifetime energy of a widely deployed model, and the marginal response to rising demand has been to provision additional GPU data-centre capacity. That response scales in capital and in power draw, and increasingly in grid-interconnection lead time; it is not obviously sustainable on the demand curve currently projected.

A separate hardware trend is underexploited in this context. UMA systems,[1] in which CPU, GPU, and neural-accelerator blocks share a single high-bandwidth memory pool, remove the host-to-device copy that dominates small-batch inference on discrete accelerators. On such hardware—Apple Silicon and Qualcomm Snapdragon X being the most widely available examples—model weights can remain resident in unified memory and be served with competitive performance per watt, particularly outside the very high batch regime for which data-centre GPUs are optimised. Despite this, the distributed-compute networks that might aggregate such hardware (Section 3) are built around heterogeneous discrete GPUs and treat neither UMA hardware nor energy efficiency as a first-class design target.

[1] Throughout, UMA denotes *Unified Memory Architecture* (a single physical memory pool shared by CPU, GPU, and accelerator blocks, as in Apple Silicon). This is distinct from the classical use of "UMA" for *uniform memory access*—the symmetric-multiprocessing model contrasted with non-uniform memory access (NUMA). The two concepts are related but not identical: a unified-memory device need not exhibit uniform access latency across all compute blocks.

This paper proposes OpenPrism Network, a distributed inference architecture that does. It is a *position and architecture* paper: we claim no original experimental results, and every quantitative figure is sourced from publicly available benchmarks or published specifications and cited as such. The contributions are as follows.

1. A UMA-first node architecture in which certified UMA devices are Tier-1 nodes, transformer layers are statically owned so that only activations transit the network, and a blockchain layer is confined to settlement and reputation rather than compute (Section 4).
2. *Idle-Capacity Harvesting*, a *capital-and-embodied* energy-cost model (Section 2) that is the basis for the distributed-mesh deployment (Model 1). We state its assumptions explicitly and are clear about what it does *not* claim: it is not an assertion that UMA hardware uses fewer joules per token than a batched data-centre GPU.
3. Two deployment models under one architecture (Section 6): a distributed mesh that harvests idle institutional capacity, and a purpose-built UMA micro data center for organisations that require a sovereign inference facility, with a total-cost-of-ownership analysis for the latter (Figure 6).
4. An honest treatment of efficiency: because output integrity costs threefold redundancy, we report performance per watt including that cost and show that UMA nodes lose operationally against batched GPUs in the scoped regime (Figure 5); we then locate the architecture's real advantage on capital, embodied energy, TCO, and sovereignty.
5. A precise statement of two unsolved problems (Section 8) that this architecture surfaces but does not claim to solve.

Scope. The design is deliberately bounded. Static layer ownership means that, in autoregressive generation, every token crosses node boundaries; the network is therefore unsuitable for low-latency, single-stream interactive inference and is scoped exclusively to batch- and throughput-oriented, latency-tolerant workloads served through asynchronous queues. We state this as a design boundary rather than a limitation to be discovered.

Stance on what is unsolved. This is an infrastructure proposal, and we have tried to keep every architectural claim technically defensible and to mark clearly where a claim ends and an open research problem begins. Section 8, including its explicit limitations subsection, bounds what the rest of the paper asserts.

2 Problem Statement

We make the gap precise as three claims, then introduce the cost model that the distributed-mesh deployment is built to exploit.

2.1 The gap

(P1) **No distributed inference network treats UMA hardware as a first-class node.** Existing networks (Section 3) admit UMA devices only as a degenerate case of a generic CPU/GPU worker, inheriting scheduling, partitioning, and verification assumptions designed for discrete accelerators. "First-class" here means that node certification, layer partitioning, and the routing fabric are designed around unified-memory residency and the bandwidth/latency profile of UMA devices—not retrofitted to it.

(P2) **Energy efficiency is not a published, first-class metric.** These networks publish throughput, price, and availability; none publishes performance per watt as a primary, methodologically pinned figure.

(P3) **The cost structure of the underlying hardware is mismodelled.** Dedicated data-centre compute and harvested institutional hardware have fundamentally different marginal-energy and capital cost structures (Section 2.3). Networks that model all supply as equivalent "compute" obscure precisely the asymmetry that makes edge and institutional deployment viable.

2.2 Why chip-level perf-per-watt is the wrong axis

It is tempting to ground the proposal in a claim that UMA silicon is more efficient per watt than a data-centre GPU. That claim is regime-dependent and, in the regime this network operates in, weak. A discrete data-centre GPU is least efficient at small batch sizes, where it is memory-bandwidth-bound and its power is amortised over little useful work; it is *most* efficient at large batch sizes. Because static layer ownership confines this network to throughput-oriented batch workloads (Section 1), the network operates in the regime most favourable to the data-centre GPU's perf-per-watt, not least favourable. Stacking the threefold redundancy of consensus (Section 4.2) on top means the UMA path must exceed a *batched* GPU by more than a factor of three on raw perf-per-watt merely to break even—a margin that, at the silicon level, does not exist in this regime (Section 5, Figure 5). The durable argument is therefore made one level up, and only for the harvested-capacity deployment.

2.3 Idle-Capacity Harvesting

Definition 1 (Idle-Capacity Harvesting)**.** *Idle-Capacity Harvesting* is the use, for inference, of computing hardware that is (a) already powered for an unrelated primary purpose, and (b) already capital- and embodied-energy-amortised by its owner, such that the marginal energy attributable to inference is the active power draw *above the device's idle baseline*, rather than its full draw.

Formally, consider a unit of inference work requiring active time t. For a dedicated data-centre GPU the system-level energy attributable to that work is

$$E_{\mathrm{dc}} = \underbrace{P_{\mathrm{gpu}}\, t}_{\text{IT load}} \cdot \underbrace{\mathrm{PUE}}_{\text{facility overhead}} + \underbrace{E_{\mathrm{emb}}\, \tfrac{t}{T_{\mathrm{life}}}}_{\text{amortised embodied/capital}}, \tag{1}$$

where $\mathrm{PUE} \geq 1$ is the facility's Power Usage Effectiveness, E_{emb} is embodied energy, and T_{life} is service life. For an idle-capacity-harvested UMA node the comparable quantity is

$$E_{\mathrm{ich}} = \left(P_{\mathrm{active}} - P_{\mathrm{idle}}\right) t, \tag{2}$$

because the idle baseline P_{idle} is consumed regardless—the device is on for its primary purpose—and embodied energy and capital are already committed and amortised against that purpose. There is no facility-overhead multiplier: an office or laboratory machine is cooled by building HVAC that operates independently of whether the device is computing.

What this does and does not claim. Equation (2) concerns the energy *attributable* to inference under harvesting; it is *not* a claim that a UMA node produces a token for fewer joules than a batched data-centre GPU. It does not: in the scoped batch regime the UMA node's throughput per watt is roughly an order of magnitude lower (Figure 5), so its operational joules per token are *higher*. The force of Idle-Capacity Harvesting is entirely in *what must be built and capitalised*: harvested capacity adds near-zero marginal capital, no new embodied energy, and no new facility, so the next increment of latency-tolerant inference can be served without constructing data-centre capacity. Idle-Capacity Harvesting is thus a property of a specific deployment—Model 1, the distributed mesh—and holds *only if the device would be powered*

anyway. If hardware is provisioned and powered specifically to serve the network, P_{idle} is no longer sunk, embodied energy is no longer amortised against another purpose, and the correct comparison reverts toward Equation (1). We therefore do not apply Idle-Capacity Harvesting to the purpose-built micro data center (Model 2, Section 6.2), whose economics rest on TCO and sovereignty instead.

3 Background

3.1 Unified Memory Architecture

In a UMA system the CPU, GPU, and dedicated matrix/neural accelerators address a single physical memory pool over a high-bandwidth fabric. For inference this removes the explicit host-to-device transfers that, on discrete accelerators, dominate latency at small batch sizes and constrain the largest model that fits without partitioning. Apple Silicon (M-series) and Qualcomm Snapdragon X are the most widely available implementations; both expose tens to over a hundred gigabytes of unified memory and sustain inference at power envelopes an order of magnitude below data-centre accelerators [6, 7]. The relevant ceiling is memory capacity: a single node holds only as large a model as its unified pool permits, which motivates the layer-partitioning of Section 4.

3.2 Inference parallelism and activation transit

Splitting a transformer across devices is well understood. Tensor parallelism partitions individual layers across devices and incurs per-layer all-reduce-style communication [3]; pipeline parallelism assigns contiguous blocks of layers to devices and passes activations forward between blocks [2]. Static layer ownership (Section 4.1) is a pipeline-style partition in which the assignment is fixed and weights never move. The cost is that, in autoregressive decoding, each generated token traverses the full pipeline and therefore every inter-node boundary; this is the mechanism behind the batch-only scope of Section 1.

3.3 Output verification

Approaches to verifying that a remote node computed an inference honestly fall into three families: cryptographic proofs of computation (e.g. zero-knowledge proofs), presently impractical for full-model LLM inference at scale; trusted-execution attestation, which moves trust into hardware enclaves and their vendors; and redundant recomputation with agreement. This proposal takes the third route, with the complication that outputs are not bitwise reproducible across heterogeneous UMA hardware—the basis of Problem I in Section 8.

3.4 Data-centre energy accounting

Power Usage Effectiveness (PUE) is the ratio of total facility energy to IT-load energy. Best-in-class hyperscale facilities approach 1.1–1.2, but the industry-wide weighted average has held near 1.54 for six consecutive years, constrained by legacy infrastructure and regional cooling limits [4]. PUE is the facility-level overhead term in Equation (1); a factor of order 1.5 does not by itself reconcile the much larger operational perf-per-watt gap between UMA nodes and batched data-centre GPUs, so the systems-level argument here does not rest on PUE alone.

3.5 Existing distributed-compute networks

Several networks aggregate third-party hardware for ML workloads [14, 15, 16, 17]. They differ in mechanism but share the assumptions this paper departs from: the unit of supply is a generic

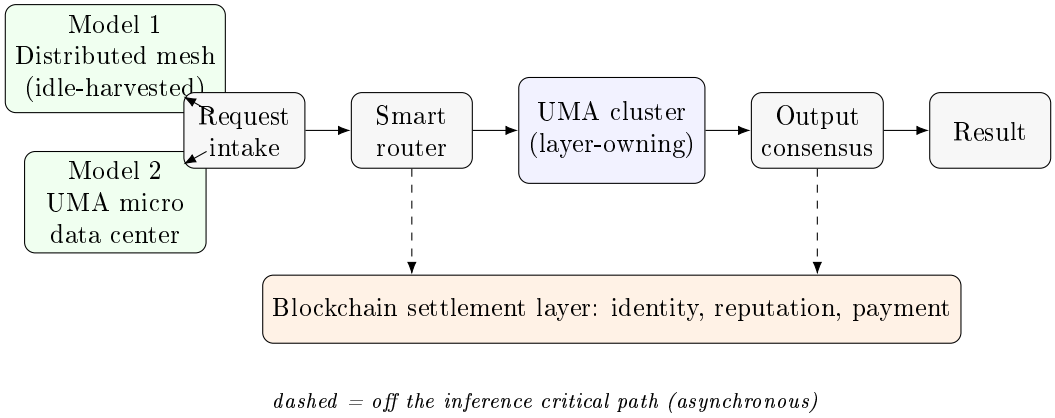

Figure 1: System architecture overview. Both deployment models (green) enter the same pipeline as alternative node populations: a request flows through intake, the smart router, a metro-local cluster of UMA nodes that jointly own the model's layers, output consensus over k redundant computations, and result return. The settlement layer (orange) records identity, reputation, and payment asynchronously and never carries activations or weights.

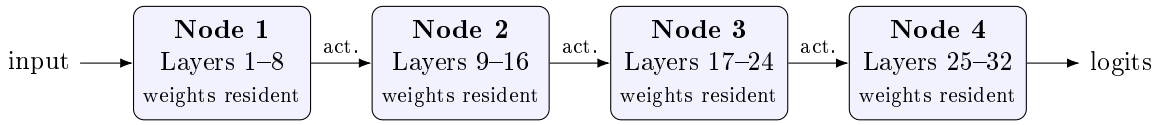

Figure 2: Node layer-ownership map for a 32-layer model partitioned across four nodes. Each node permanently holds the weights for a contiguous layer block; only activations ("act.") flow between nodes. The same partition is what lets a model exceed any single node's unified-memory capacity, at the cost of one inter-node hop per layer boundary per token.

discrete GPU, verification and scheduling are designed around that unit, and energy efficiency is not a first-class published metric. Section 7 draws the comparison precisely (Figure 7).

4 Proposed Architecture

OpenPrism serves inference requests by routing them to clusters of certified UMA nodes that collectively hold a model, establishing output integrity by redundant agreement, and settling payment and reputation on a separate blockchain layer that is never in the inference data path. Both deployment models (Section 6) enter the same architecture as alternative node populations; Figure 1 shows the end-to-end flow.

4.1 Static layer ownership

A model is partitioned pipeline-wise into contiguous blocks of transformer layers, and each block is "owned" by a node on which those weights remain permanently resident. Only activations transit the network between successive owners; weights never move during inference (Figure 2). Static ownership exploits unified memory directly (a node computes from the same pool its block resides in, with no host-to-device staging), bounds each node's memory requirement to its block rather than the whole model, and—this is the cost—in autoregressive decoding forces every generated token to traverse every inter-node boundary, which is the mechanism behind the batch-only scope of Section 1.

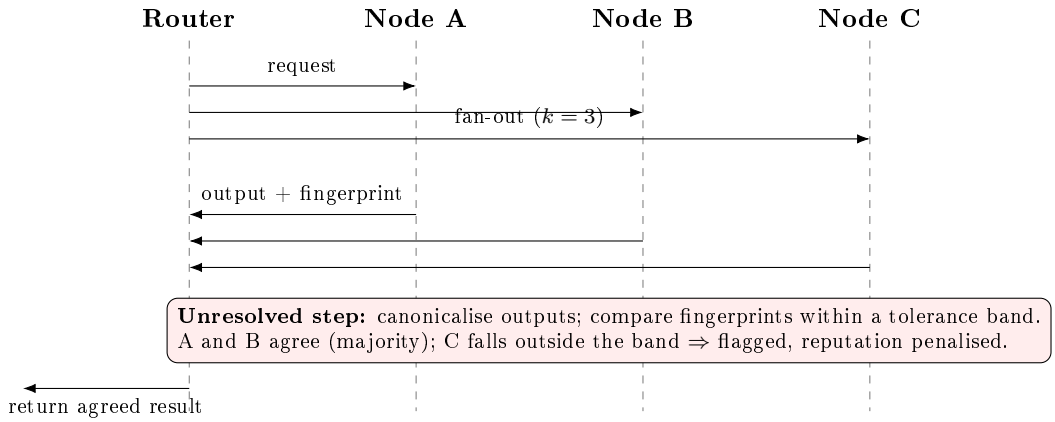

Figure 3: Output consensus protocol flow. The router fans a request out to $k = 3$ nodes; each returns its output and a canonicalised fingerprint; the router accepts the majority-agreeing result within a tolerance band and penalises the dissenting node. The tolerance-band comparison (red)—robust to honest floating-point variation yet sensitive to genuine divergence—is the unresolved step of Section 8.1.

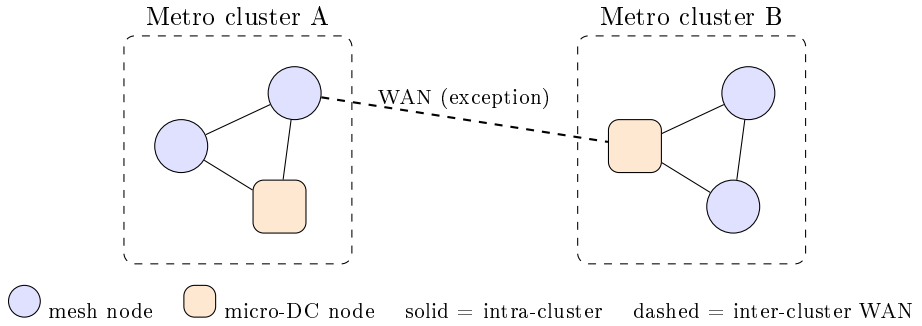

Figure 4: Locality-aware routing topology. Inference is kept within a metro cluster over low-latency intra-cluster links (solid); inter-cluster WAN links (dashed) are used only when a single cluster cannot host the model. Mesh nodes (circles) and micro-data-center nodes (squares) may co-populate a cluster.

4.2 Output verification by redundant agreement

Each request is computed on k independent nodes (or node-paths); their outputs are reduced to canonicalised fingerprints and compared, and a result is returned only if a majority agree (Figure 3). We use $k = 3$ as the default. This is the source of the threefold energy cost reported honestly in Section 5, and its hard technical core—agreement under floating-point nondeterminism across heterogeneous UMA hardware—is the open problem of Section 8.1, marked as the unresolved step in the figure.

4.3 Locality-aware routing

Because activation transit is the dominant non-compute cost, the router treats network locality as a first-order scheduling constraint. Nodes are grouped into metro-area clusters; the router places a model's layer blocks within a single cluster wherever possible, so that activation hops stay on low-latency intra-metro links. Wide-area (WAN) links between clusters are the exception, reserved for cases where no single cluster has the aggregate memory or free capacity to host a model (Figure 4). Mesh nodes (Model 1) and micro-data-center nodes (Model 2) appear as distinct node types that may co-populate a cluster.

4.4 Asynchronous queuing and the failure model

Requests enter an asynchronous queue rather than a synchronous request/response path. This matches the scoped workload and makes the network robust to the node churn inherent in harvested institutional hardware: a node that drops out mid-queue causes its in-flight work to be re-dispatched rather than failing the request. The network is designed for failure and redundancy from the outset; node unavailability is the expected case, not an exception.

4.5 Settlement layer

A blockchain records node identity, accumulates reputation from verification outcomes, and settles payment. It is deliberately *not* a compute or consensus substrate for inference: no activations, weights, or model outputs are placed on-chain, and the chain is never in the critical path of serving a request. Keeping settlement off the data path is what allows the inference fabric to be evaluated purely on its own latency and energy characteristics, independent of any blockchain's throughput.

4.6 Trust and threat model

The network assumes a partially adversarial node population. (1) *Honest majority within a verification group:* redundant agreement (Section 4.2) detects a faulty minority but fails if a majority of a request's k replicas collude on the same wrong answer; with $k = 3$ the scheme tolerates a single Byzantine replica per group (fraction $< 1/2$ within the group). Reputation scoring and randomised replica assignment raise the cost of achieving a colluding majority but do not eliminate it. (2) *Sybil resistance via the settlement layer:* identity and stake on-chain raise the cost of flooding the network with cheap fake nodes, which is what protects the randomness of replica assignment. (3) *No confidentiality guarantee against a node operator:* a node sees the activations it processes, so the design does not by itself protect input privacy from a malicious owner—relevant to the institutional-deployment audience (Section 6) for whom on-premises trust boundaries, not cryptographic input privacy, are the operative control. Confidential-computing extensions are out of scope.

5 Hardware Target Profile

A node qualifies as Tier-1 if it meets unified-memory, bandwidth, and sustained efficiency thresholds appropriate to layer-resident inference. Representative criteria: a unified-memory pool sufficient to hold a useful contiguous layer block with KV-cache headroom (in practice $\geq 64\,\text{GB}$); sustained memory bandwidth in the hundreds of GB/s (Apple M3 Ultra provides $\approx 819\,\text{GB/s}$ [6]); and a sustained, measured tok/s/W above a published floor for the certified model class. Devices that cannot sustain bandwidth under thermal load, or that lack unified memory (discrete-accelerator hosts staging weights over PCIe), are disqualified from Tier-1.

Figure 5 reports the efficiency picture honestly. The point is not that UMA nodes win on operational efficiency—in the scoped batch regime they do not—but that the figure is reported transparently and the architecture's advantage is located elsewhere (Equation (2), Figure 6).

6 Deployment Models

The same architecture (Section 4) supports two deployment models with *distinct* economics. We keep them strictly separate: the Idle-Capacity Harvesting argument applies to Model 1 only.

Platform (70B, 4-bit)	**Chip-level** tok/s/W (raw)	**Systems-level** tok/s/W (PUE-adj.)
UMA Tier-1 node (M3 Ultra)	0.085	0.097†
Batched H100 (FP8, batch ≈ 32)	2.7	1.75
Batched H100 (MLPerf max)	4.3	2.8

Figure 5: Power-efficiency comparison (operational). *Chip-level* divides throughput by device power; *systems-level* folds in facility overhead. Assumptions: UMA node ≈17 tok/s at ≈200 W on 70B-Q4 [8, 9]; H100 ≈ 1900 tok/s (FP8, batch ≈ 32) and MLPerf-class peak on a 700 W part [10, 5]; data-centre PUE = 1.54 [4]. †For the mesh (Model 1) the systems-level figure uses marginal draw (active − idle) with no PUE multiplier per Equation (2); the data-centre figure divides chip-level by PUE. UMA nodes lose operational efficiency by roughly an order of magnitude even after PUE; the architecture's advantage is capital and TCO (Figure 6), not operational joules per token. *Figures derived from published specifications and community benchmarks; original experimental validation is future work.*

6.1 Model 1: Distributed Inference Mesh

Model 1 harvests idle capacity across existing institutional UMA hardware—laboratory workstations, teaching-cluster Macs, biotech analysis machines—that is powered for a primary purpose and idle outside working hours. Under Idle-Capacity Harvesting (Definition 1, Equation (2)) the marginal energy attributable to inference is the active draw above an idle baseline the institution already pays, and the marginal *capital* is approximately zero: no hardware is purchased, no facility is built, no embodied energy is newly incurred. Model 1 does not claim lower operational joules per token than a batched GPU (Figure 5); it claims that the next increment of latency-tolerant inference capacity can be stood up at near-zero capital and without new construction. Its target deployers are universities, research labs, and biotech workstations with existing fleets and latency-tolerant workloads.

6.2 Model 2: UMA Micro Data Center

Model 2 is a purpose-built, UMA-first, locally owned inference facility—on the order of a single rack of Apple Silicon nodes, with no discrete GPUs— deployable by an organisation that requires a sovereign inference facility. Because the hardware is provisioned specifically for inference, Idle-Capacity Harvesting does *not* apply: capital is real and energy is paid in full (though without a data-centre PUE multiplier, since a single rack uses existing building cooling). Model 2's case rests on two grounds. First, *data sovereignty*: inference runs on-premises, so organisations that cannot send data to hyperscalers for regulatory reasons (hospital systems, government agencies, regulated biotech) or cannot afford hyperscaler pricing (emerging-market institutions) retain control. Second, *total cost of ownership* at realistic utilisation, quantified in Figure 6.

7 Differentiation

Figure 7 contrasts OpenPrism with the principal existing networks. The distinguishing commitments are a UMA-first node architecture, energy and cost efficiency reported as first-class (and honest) published metrics, an institutional/edge target audience rather than a crypto-native one, and a blockchain confined to settlement.

	60% sustained	30% bursty
UMA Micro Data Center (8 × M3 Ultra, owned)		
Capital: hardware + 10GbE + housing	$33,992 (one-time)	
Amortised capital / month (36 mo)	$944	$944
Electricity / month ($0.14/kWh, no PUE)	$106	$63
Subtotal / month	**$1,050**	**$1,007**
3-year total	**$37,800**	**$36,300**
AWS p4d.24xlarge — equivalent 70B-Q4 token volume		
On-demand / month ($21.96/hr)	$1,634	$817
3-year total (on-demand)	**$58,800**	**$29,400**
1-yr reserved / month (dedicated)	$9,485	$9,485
Lower 3-year cost	**UMA owned**	**Cloud on-demand**

Figure 6: Total cost of ownership: a UMA Micro Data Center (8 × M3 Ultra) versus AWS `p4d.24xlarge` (8 × A100, us-east-1) sized to deliver the same sustained 70B-Q4 token volume, 3-year horizon. The honest result is a crossover, not a single winner: **owned UMA is cheaper at high sustained utilisation (60%); cloud on-demand is cheaper when bursty (30%); break-even is ≈ 38% utilisation.** Cloud prices are published list prices [12]; on-demand $21.96/hr, 1-yr reserved $9,485/mo. UMA hardware at $3,999/node [6]. Throughput equivalence uses the mesh aggregate (8× ≈ 17 tok/s) against an estimated p4d 70B-Q4 throughput (≈ 800 tok/s); reserved cloud is rational only near continuous use and is shown for completeness. *TCO is illustrative, based on published pricing; staff/administration cost is excluded and is a real variable; the `p5.48xlarge` (8 × H100, $98.32/hr [13]) shifts the cloud column upward. Actual costs vary by deployment.*

8 Open Research Problems

Two problems are central to this architecture and, to our knowledge, unsolved. We state them as research challenges, not as components we have built.

8.1 Problem I: consensus for ML output verification under floating-point non-determinism

The verification scheme of Section 4.2 compares the outputs of k independent computations and accepts on majority agreement (Figure 3). The difficulty is that "agreement" is not exact-match: the same model, given the same input, does not produce bitwise identical logits across heterogeneous UMA hardware—different silicon generations, accelerator kernels, and reduction orders yield small floating-point differences. Naive cryptographic fingerprinting (hash and compare) therefore fails, because honest nodes disagree at the bit level. The open problem is a consensus protocol that (a) canonicalises outputs into a representation stable across honest heterogeneous hardware, (b) defines a tolerance band wide enough to admit honest floating-point variation yet narrow enough to reject a materially different or fabricated result, and (c) is cheap enough that verification does not dominate. This is distinct from general blockchain consensus, which concerns agreement on an ordered ledger, not on whether two high-dimensional floating-point tensors are "the same computation."

	OpenPrism	**Gensyn**	**Bittensor**	**io.net**	**Akash**
Node architecture	UMA-first (Apple Silicon / Snapdragon X)	GPU-generic	GPU-generic	GPU-generic (aggregated)	GPU-generic + general compute
Verification model	Redundant agreement + tolerance-band fingerprint (open)	Verified compute / proof-of-learning	Subnet incentive consensus	SLA / limited	Marketplace SLA (no ML-specific)
Energy efficiency published?	Yes (perf/W + TCO, honest)	No	No	No	No
Target audience	Institutions / edge (universities, biotech, hospitals, gov.)	ML & crypto compute	Crypto-native ML markets	ML teams seeking cheap GPU	General cloud users
Deployment model	Mesh + micro data center	Decentralised compute protocol	Subnet network	DePIN GPU cloud	Decentralised cloud marketplace
Blockchain role	Settlement / reputation only (off path)	Verification + settlement	Core (consensus, incentive, token)	Coordination / payment	Marketplace settlement / bidding

Figure 7: Differentiation of OpenPrism Network against existing distributed-compute networks. Characterisations of other projects are drawn from their public documentation [14, 15, 16, 17].

8.2 Problem II: dynamic layer reassignment without full weight redistribution

Static layer ownership (Section 4.1) is efficient while membership is stable, but harvested institutional nodes join and leave continually. When a node owning layers i through j departs, those layers must be served by another node—which, naively, requires transferring their weights across the network, exactly the bulk movement static ownership was designed to avoid. The open problem is a reassignment protocol that rebalances ownership as membership changes while keeping weight movement sublinear in model size: for example via pre-staged replicas on a bounded number of standby nodes, erasure-coded layer shards reconstructable from survivors, or overlapping ownership that degrades gracefully. Each candidate trades memory or redundancy against rebalancing cost, and none is obviously optimal under realistic churn.

8.3 Limitations

Beyond the two open problems, the architecture has limitations we state plainly. *Latency:* the batch-only scope (Section 1) excludes interactive single-stream use entirely. *Operational efficiency:* UMA nodes lose to batched data-centre GPUs on joules per token in the scoped regime (Figure 5); the Idle-Capacity Harvesting and TCO arguments do not overturn this and do not apply to Model 2's energy. *Verification cost:* $k = 3$ redundancy triples compute. *Privacy:* no confidentiality against a node operator (Section 4.6). *Empirical status:* this is a position and architecture paper; all figures derive from published specifications and community benchmarks, and end-to-end experimental validation—including measured tolerance-band behaviour and churn-rebalancing cost—is future work.

9 Conclusion

OpenPrism Network is a UMA-first distributed inference architecture for latency-tolerant, batch-oriented workloads, offered in two deployment models: a mesh that harvests idle institutional capacity at near-zero marginal capital, and a purpose-built micro data center for sovereign, on-premises inference. We have been deliberate about what is and is not claimed: UMA nodes do not win on operational efficiency in the scoped regime, and we say so; the architecture's case rests on capital, embodied energy, total cost of ownership at sustained utilisation, and data sovereignty. Two problems remain genuinely open—consensus for ML output verification under floating-point non-determinism, and dynamic layer reassignment without full weight redistribution—and we invite contributions on both.

Call to contribute. The architecture specification, reference interfaces, and the Mermaid sources for the diagrams in this paper are maintained at the project repository (`https://github.com/openprism-network/openprism`; placeholder pending release). This paper is archived at Zenodo, DOI 10.5281/zenodo.20465112. Contributions are particularly welcome on the two open problems of Section 8.

References

[1] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] Y. Huang et al., "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism," *NeurIPS*, 2019.

[3] M. Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," arXiv:1909.08053, 2019.

[4] Uptime Institute, "Global Data Center Survey 2025," 2025. Weighted-average annual PUE reported at approximately 1.54.

[5] MLCommons, "MLPerf Inference: Datacenter," benchmark results (v4.1), `mlcommons.org`, 2024.

[6] Apple Inc., "Mac Studio (M3 Ultra) — Technical Specifications," `apple.com`, 2025–2026 (unified memory, ≈819 GB/s memory bandwidth, pricing).

[7] ggml-org, "Performance of llama.cpp on Apple Silicon M-series," community benchmark discussion #4167, `github.com`.

[8] Hostbor, "Mac Studio M3 Ultra Tested," 2025 (local 70B-class inference throughput and at-the-wall power).

[9] Private LLM, internal Apple Silicon Llama-3 70B throughput datapoints, 2024.

[10] Spheron, "Tokens-per-Watt for AI Inference," 2026 (H100 70B FP8 throughput at batch).

[11] NVIDIA, "H100 Tensor Core GPU — Datasheet," `nvidia.com` (≈700 W TDP).

[12] Amazon Web Services, "Amazon EC2 P4 Instances — Pricing" (`p4d.24xlarge`, us-east-1; on-demand $21.96/hr, 1-yr reserved), `aws.amazon.com`, accessed 2026.

[13] Amazon Web Services, "Amazon EC2 P5 Instances — Pricing" (`p5.48xlarge`, $98.32/hr), `aws.amazon.com`, accessed 2026.

[14] Gensyn, technical documentation / litepaper, `gensyn.ai`.

[15] Bittensor, "Bittensor Whitepaper" and developer documentation, `bittensor.com`.

[16] io.net, network documentation, `docs.io.net`.

[17] Akash Network, documentation, `akash.network`.